Implementation Companion

Python Examples for Cyber Control, Differential Games, RL/MADRL, PINN, and PIDL

From mathematical formulation to reproducible experiments

Companion to Lecture Notes 1 and 2

Contents

1. Purpose

This companion explains the accompanying Python files. The goal is not to provide production-ready benchmark software. The goal is to make the conversion from equations to code explicit and extendable. Each script is short enough to read, but structured enough to become a research prototype.

2. File Map

File	Role
`cyber_dynamics.py`	shared ODE and RK4 utilities; controlled SIR and hybrid RHS functions
`cyber_hybrid_env.py`	sampled-data environment with continuous flow, jump map, and hybrid actions
`fbsm_malware_baseline.py`	open-loop FBSM baseline from PMP
`ddqn_cyber_defense.py`	discrete-action DDQN defender against a scripted attacker
`madrl_ctde_hybrid_game.py`	compact CTDE/MADRL attacker-defender example
`inverse_pinn_sir_malware.py`	inverse PINN for learning β and γ from sparse $I(t)$ data
`pidl_unknown_mechanism.py`	known SIR mechanism plus learned missing correction term
`control_pinn_malware.py`	direct neural-control PINN
`pmp_informed_pinn_malware.py`	state-costate-control PMP-informed PINN

3. General Implementation Architecture

A clean implementation should separate four layers:

1. model layer: ODE right-hand side, parameters, constraints;
2. environment layer: reset, action decoding, jump map, ODE integration, reward;
3. algorithm layer: FBSM, DDQN, MADRL, PINN, PIDL;
4. evaluation layer: baselines, robustness, plots, logs, seeds.

The code folder follows this structure. The environment is intentionally independent of DQN, PPO, or MADRL so that different learning algorithms can reuse the same cyber model.

4. ODE and Hybrid Environment

The central environment step is

$$\boxed{s_k} \to \boxed{a_D^k, a_A^k} \to \boxed{x_k^+ = G(x_k, a_k)} \to \boxed{x_{k+1} = \mathrm{RK4}(f, x_k^+)} \to \boxed{r_D^k, r_A^k}. \tag{1}$$

The implementation is in `cyber_hybrid_env.py`. The most important design choice is to keep jump effects separate from continuous flow effects.

Listing 1: Core idea of a sampled-data hybrid environment step.

```
def step(self, defender_action, attacker_action):
    dpar = self.defense_parameters(defender_action)
    apar = self.attack_parameters(attacker_action)
    pre_jump = self.state.copy()
    post_jump = self.jump_map(pre_jump, dpar)
    rhs = lambda x, t: hybrid_rhs(x, dpar, apar, self.cfg.params)
    next_state, path = rk4_integrate(rhs, post_jump, t0=self.t*self.cfg.dt,
                                dt=self.cfg.dt, substeps=self.cfg.substeps,
                                project=project_simplex3)
    reward = compute_reward(path, dpar, apar)
    return next_state, reward, done, info
```

4.1 How to extend this environment

To extend from the simple $[S, I, R, z]$ model to a degree-based network model, replace the state vector with

$$x = [S_1, I_1, R_1, \ldots, S_K, I_K, R_K, z] \tag{2}$$

where K is the number of degree classes. Then update the RHS so that infection pressure for degree class k uses a degree-weighted infected neighbor probability. The RL environment interface does not change.

5. FBSM Baseline

The FBSM file solves a continuous-control malware problem. It should be used as a classical baseline when the model is known and the control is open-loop. It is not a substitute for DDQN or MADRL because it does not learn a feedback policy.

Listing 2: FBSM update logic.

```
for iteration in range(max_iter):
    # forward state integration using current u(t)
    for k in range(n):
        x[k+1] = rk4_state_step(x[k], u[k], h, beta, gamma)
    # backward costate integration from terminal condition
    lambda[n] = terminal_gradient
    for k in range(n, 0, -1):
        lambda[k-1] = rk4_costate_step_backward(x[k], lambda[k], u[k], h)
    # stationarity update and relaxation
    u_new[k] = clip(S[k]*(lambda_S[k]-lambda_R[k])/B, 0, u_max)
    u = relax*u_new + (1-relax)*u
```

6. DDQN Example

The DDQN example trains the defender with discrete actions. The attacker is scripted so that the defender first learns against a stable environment. This is the recommended first step before full MADRL.

DDQN workflow.

1. initialize Q-network and target Q-network;
2. collect transitions (s, a, r, s', d) using epsilon-greedy exploration;
3. store transitions in replay buffer;
4. sample mini-batches;
5. compute DDQN target;
6. update the online network;
7. periodically copy online weights to target weights;
8. validate against static and rule-based policies.

7. MADRL / CTDE Example

The MADRL file uses decentralized actors and centralized critics. It is intentionally compact. The defender and attacker both receive the same state observation in the example, but this can be changed to partial observations.

7.1 General extension to richer MADRL

For serious experiments, add:

- generalized advantage estimation (GAE);
- clipped PPO ratios for MAPPO;
- opponent pools to reduce cycling;
- multiple seeds and cross-play matrices;
- best-response retraining as an approximate exploitability test;
- separate observation functions for attacker and defender.

8. Inverse PINN Example

The inverse PINN learns β and γ from sparse observations of $I(t)$. It uses softmax for the state network and softplus for positive parameters.

Listing 3: PINN residual construction.

```
x_f = model(t_f)
dxdt = torch.cat([
    torch.autograd.grad(x_f[:, j].sum(), t_f, create_graph=True)[0]
    for j in range(3)
], dim=1)
loss_ode = torch.mean((dxdt - sir_rhs(x_f, beta, gamma))**2)
loss = loss_data + w_ic*loss_ic + w_ode*loss_ode
```

9. PIDL Missing-Mechanism Example

The PIDL script uses

$$\dot{x} = f_{\text{known}}(x; \beta, \gamma) + B g_{\phi}(x). \tag{3}$$

The correction network learns only the missing transfer from S to I, not the entire vector field. This is preferable when the known mechanism is trusted but incomplete.

10. Control PINN and PMP-informed PINN

10.1 Direct control PINN

The direct control PINN minimizes a control objective plus state residuals. It is easy to code and useful for teaching.

10.2 PMP-informed PINN

The PMP-informed PINN adds costate and stationarity residuals. This is the neural analogue of solving the PMP optimality system:

$$\mathcal{L} = w_x \|\dot{x} - f\|^2 + w_\lambda \|\dot{\lambda} + H_x\|^2 + w_u \|H_u\|^2 + w_b \mathcal{L}_{\text{boundary}}. \tag{4}$$

Use this formulation when the paper claims a strong connection to PMP.

11. Evaluation Scripts to Add in Future Work

The current scripts print losses and returns. A research version should save:

- CSV trajectories for S, I, R, z;
- learned parameter histories;
- training curves;
- baseline comparison tables;
- robustness results across parameter perturbations;
- policy action distributions over time.

12. Complex Extensions

12.1 Degree-based HODGE-style model

To implement a degree-based propaganda model, let the state contain P_k and C_k for each degree k. The force of influence uses

$$\Theta_P(t) = \frac{\sum_k k p_k P_k(t)}{\sum_k k p_k}, \qquad \Theta_C(t) = \frac{\sum_k k p_k C_k(t)}{\sum_k k p_k}. \tag{5}$$

Continuous propaganda investment changes ODE rates, while impulsive counter-propaganda applies a jump map at selected epochs. The same environment structure still applies.

12.2 Feedback control PINN

Change $u(t)$ to $u(x,t)$:

$$\hat{u}_\Phi(t) = N_\Phi(t) \quad \longrightarrow \quad \hat{u}_\Phi(x,t) = N_\Phi([x,t]). \tag{6}$$

Then train on many initial conditions. This is closer to a feedback policy and more comparable to RL.

13. Suggested Project Sequence

1. Run FBSM and DDQN on the same simple malware model.
2. Add hybrid actions and compare no defense, static patch, rule-based, DDQN.
3. Add attacker learning and compare independent learning with CTDE.
4. Learn unknown parameters using inverse PINN.
5. Add PIDL correction for missing mechanisms.
6. Add PMP-informed PINN and compare with FBSM.
7. Extend to APT or misinformation dynamics.